

ITEC 116

System integration

System

is simply a group of parts that work together to form a whole. Each part may have its own job, but when combined, they achieve something bigger.

Subsystems

A smaller system inside a bigger system.

It has its own purpose but also contributes to the larger system.

Component

The individual parts that make up a subsystem or system.

System Integration

is the process of joining software and hardware modules into one cohesive infrastructure. The goal is to make these systems work together smoothly so they can share information and processes more efficiently.

Integration

is the act of bringing together smaller components or information stored in different subsystems into a single functioning unit.

Types of system integration

- Vertical integration
- Horizontal integration
- Point to Point integration
- Common data integration
- Star integration

Vertical integration

Systems are integrated by function, like stacking them in layers (bottom to top or top to bottom)

Horizontal integration

Systems are integrated through a central hub or middleware, which allows different applications to communicate without needing direct connections.

Point to point integration

Each system connects directly to another system it needs to exchange data with.

Star integration

Every system connects to every other system it needs to talk to

Common data format integration

Different systems use a shared standard format so data can move more easily between them

System integration method

These are the techniques used to connect systems:

- API (Application Programming Interface)
- Webhooks
- Middleware
- Direct/Point to Point Connections

API (Application Programming Interface)

Lets apps talk to each other. Most modern integrations rely on APIs

Webhooks

Event driven communication where one system automatically pushes data to another when something happens

One system sends an automatic message to another when something happens

Middleware

Software that sits in the middle, acting as a translator

Useful when systems don't naturally understand each other

Direct/Point to Point Connections

Systems are directly connected with custom links

Benefits of system integration

- Efficiency and productivity
- Real-time data sharing
- Improved customer/user experience
- Better decision making

Limitations of system integration

- High cost
- Security risks
- Complexity
- Downtime issues

API: APPLICATION PROGRAMMING INTERFACE

APPLICATION PROGRAMMING INTERFACE (API)

A defined set of rules (interface) that developers (programming) use so one software application can interact with another.

Key components:

- Endpoint
- Request
- Response
- HTTP method
- Status Code

ENDPOINT

URL where the API is accessed.

A **URL** (Uniform Resource Locator, also called a web address) is a unique identifier used to locate a resource on the internet. URLs consist of multiple parts -- including a **protocol** and **domain name** -- that tell web browsers how and where to retrieve a resource.

URL (Uniform Resource Locator)

is the address you type in a browser to reach a specific resource on the internet—like a web page, image, or file.

Request

is a message sent by a client application to a server to ask for a specific service, functionality, or data.

The API (Application Programming Interface) acts as a messenger, processing the request and delivering the server's response back to the client.

Core components of an API request

- Endpoint
- HTTP Method
- Headers
- Body
- Parameter

Response

is the server's reply to an API request, containing information about whether the request was successful and any requested data. It's the server's way of communicating the outcome and providing the necessary information to the client application.

HTTP Method

is a component of an HTTP request that specifies the action the client wants to perform on a resource identified by a Uniform Resource Locator (URL).

In the context of RESTful APIs, HTTP methods correspond to the basic data operations: Create, Read, Update, and Delete (CRUD).

- GET
- POST
- PUT
- PATCH
- DELETE

HTTP status code

is a three-digit number issued by a server in response to a client's request to access a resource on the internet. The code indicates the outcome of the request and whether it was successful, redirected, or failed due to an error on the client or server side.

- 1xx: informational
- 2xx: success
- 3xx: Redirection
- 4xx: client error
- 5xx: server error

Types of APIs

- Public API
- Internal API
- Partner API

PUBLIC (OPEN) API

These are publicly available for use by any outside developer or business, often with minimal restrictions, though registration or an API key might be required.

INTERNAL (PRIVATE) API

These APIs are designed for internal use within an organization, connecting different systems and data sources within the enterprise to improve communication and efficiency among internal teams.

PARTNER API

These APIs are accessible only to authorized external developers, facilitating business-to-business partnerships and enabling data sharing or service integration between specific organizations.

COMMON ARCHITECTURE API

- SOAP API
- REST API
- gRPC API
- GraphQL

SOAP API

Simple object access protocol (SOAP) maintained by the World Wide Web Consortium (W3C), is one of the earliest API architectures. It utilizes XML for message formatting, ensuring a standardized approach to communication between applications.

REST API

APIs built on the **representational state transfer (REST)** architecture are popular among developers due to their simplicity.

gRPC API

gRPC, developed by **Google**, is an API architecture style that uses remote procedure calls for API interaction. Services expose functionalities as methods; clients can call them similar to local function calls.

gRPC employs Protocol Buffers, a language-neutral format for defining data structures and messages. It enables efficient communication between diverse programming languages. Like REST, it supports single request, single response communication, but being a more modern API style, it also has built-in support for server-streaming (continuous server updates), client-streaming (sending a stream of data), and bidirectional streaming (real-time communication).

GRAPHQL API

GraphQL architecture allows clients to specify the exact data they need. This is achieved through queries written in a flexible language like GraphQL Schema Language (SDL).

GraphQL queries are declarative, specifying the data needed but not how it's retrieved. This allows the server to optimize execution and leverage different data sources efficiently.

GraphQL also provides a structured error-handling mechanism that returns detailed information about errors encountered during query execution.

Monolith, Modular Monolith, and Microservices

MONOLITH

A monolithic architecture represents the traditional approach to software development. In this model, the entire application is built as a single, unified unit with a single codebase. All components, whether related to user interface, business logic, or data access, are tightly coupled and interdependent. The entire system is then compiled and deployed as a single executable file or directory. This structure is analogous to a historic building carved from a single, massive rock formation. All of its "rooms" (functions) are part of the same base, and altering one part can have an unintended impact on another.

The core characteristics of this model are its singular codebase, a unified deployment process, and an inherent tight coupling between its various components. This interdependence means that a small change in one area can affect large parts of the code base, making modifications time-consuming and prone to risk.

It is often observed that monolithic applications become complex and challenging to update over time. This is not necessarily an inherent flaw of the architecture itself, but rather a symptom of a lack of architectural discipline. Without a deliberate effort to separate concerns and create clear internal boundaries, a monolithic codebase can degrade into what is colloquially known as a "ball of mud." The problems commonly attributed to monoliths, such as slow development and difficult maintenance, are often the downstream effects of this internal disorganization, not the single-unit deployment itself.

MODULAR MONOLITH

The modular monolith is a sophisticated evolution of the traditional model, designed to address its primary shortcomings without

adopting the full complexity of a distributed system. It is a hybrid approach where the application is broken down into smaller, independent modules. While still packaged and deployed as a single unit, these modules are loosely coupled and communicate only through well-defined interfaces.

The central principle of the modular monolith is logical separation within a physical single-unit deployment. Each module encapsulates a specific business domain, with its own logic and data. Communication between modules is restricted to public APIs or service layers, preventing direct access to a module's internal implementation details. This practice reduces the risk of unintended side effects and makes the codebase more legible and easier to maintain. This architectural style achieves a high degree of modularity and organization while retaining the simplicity of a single application.

Microservices

Microservices architecture is an architectural pattern that decomposes a large application into a collection of small, independent, and loosely coupled services. Each service is built to perform a single, specific function and operates as a standalone unit with its own codebase, business logic, and often, its own database. These services communicate with each other through a well-defined interface, such as a REST API or event-driven messages.

The defining characteristics of microservices are their independent operation and deployment. Because each service is a self-contained entity, it can be developed, updated, and deployed independently of other services in the system. This independent lifecycle is a core tenet of the architecture and is what enables the high degree of agility and flexibility often associated with it. This model is a response to the inherent limitations of monolithic applications at scale.

MVC (MODEL VIEW CONTROLLER)

MVC

is a design pattern that separates an application into three interconnected components.

This separation helps manage complex applications by organizing code and improving modularity, maintainability, and scalability.

Model

Represents the data and business logic of the application.

It communicates with the database or external APIs to fetch, update, and manipulate data.

It notifies the View when data changes.

View

Responsible for the user interface (UI) and displaying data to the user.

It listens for updates from the Model and updates the UI accordingly.

The View is typically passive, meaning it doesn't contain logic beyond UI updates.

Controller

Acts as a mediator between the Model and the View.

It handles user input, interacts with the Model, and determines which View to display.

The Controller processes data before passing it to the View.

Flow of MVC

1. The user interacts with the View (e.g., clicking a button).
2. The View sends the input to the Controller.
3. The Controller processes the input and interacts with the Model.

4. The Model updates its data and notifies the View of any changes.

5. The View refreshes to reflect updated data.

CONTROLLER SERVICE REPOSITORY

The Controller→ Service→ Repository pattern builds on the MVC concept by introducing a more layered architecture, which is especially useful in backend development. Each layer has a distinct role, improving separation of concerns and maintainability.

Controller

The Controller handles HTTP requests and responses.

It validates and processes incoming requests, then delegates the business logic to the Service layer.

The controller typically does not contain business logic.

Service

The Service layer contains the business logic of the application.

It performs complex operations, applies rules, and interacts with multiple repositories if needed.

This layer is reusable and modular.

Repository

The Repository layer handles data access and communication with the database.

It abstracts database operations, making it easier to switch data sources if needed.

Repositories typically use Object-Relational Mapping (ORM) tools like TypeORM,

Flow of execution

1. Client sends a request to the Controller (e.g., GET /users/:id).

2. The Controller delegates the logic to the Service layer.
3. The Service calls the Repository to fetch data from the database.
4. The Repository interacts with the database and returns the result to the Service.
5. The Service processes the result and returns it to the Controller.
6. The Controller sends the final response to the client.

Object Relational Mapping (ORM)

Object Relational Mapping(ORM)

Object Relational Mapping (ORM) is a technique used in creating a "bridge" between object-oriented programs and, in most cases, relational databases.

What is an ORM tool?

An ORM tool is software designed to help OOP developers interact with relational databases. So instead of creating your own ORM software from scratch, you can make use of these tools.

Popular ORM Tools for Java

1. Hibernate
2. Apache OpenJPA
3. EclipseLink
4. jOOQ
5. Oracle TopLink

Popular ORM Tools for Python

1. Django
2. web2py
3. SQLAlchemy
4. SQLAlchemy

5. Oracle TopLink

Popular ORM Tools for PHP

1. Laravel
2. CakePHP
3. Qcodo
4. RedBeanPHP
5. Oracle TopLink

Popular ORM Tools for .NET

1. Entity Framework
2. NHibernate
3. Dapper
4. Base One Foundation Component Library (BFC)